Estado da Arte

How do different programming languages solve the tail call optimization problem?

Seu Nome
Pontifícia Universidade Católica do Paraná
3 de maio de 2026

1 Introdução

A otimização de chamadas em cauda (*Tail Call Optimization* — TCO) é uma técnica compilacional que permite reutilizar o quadro de pilha (*stack frame*) da função chamadora quando a chamada a uma subrotina é a última ação executada por essa função (STEELE JR., 1977). Sem essa otimização, funções recursivas em cauda consomem espaço de pilha proporcional ao número de chamadas — complexidade $O(n)$ —, podendo ocasionar estouro de pilha (*stack overflow*) em entradas de grande porte. Com TCO, o consumo é reduzido para $O(1)$ (WIKIPEDIA CONTRIBUTORS, 2025).

A recursão em cauda transcende a mera economia de espaço, estabelecendo o alicerce para representações avançadas de compiladores, como o Estilo de Passagem de Continuação (Continuation-Passing Style - CPS). Além disso, este levantamento aborda como transformações de fluxo transferem a responsabilidade do hardware para o Coletor de Lixo (GC) e avalia o impacto destas técnicas sob a ótica de microbenchmarking e sustentabilidade energética (GreenScore).

2 Tail Call Optimization: Fundamentos e Semântica

Uma *tail call* ocorre quando a chamada a uma função é a última operação de outra função, sem nenhum cálculo posterior sobre o resultado retornado (STEELE JR., 1977). A literatura distingue a TCO (uma otimização oportunista) da *Proper Tail Recursion* (PTR), que é uma garantia semântica de espaço assintótico constante (BIERNACKA; DANVY, 2005).

Duas técnicas principais são empregadas na implementação e mitigação de TCO:

- **Substituição direta e CPS:** O compilador detecta a chamada em cauda e emite um salto (*jump*). No paradigma de *Continuation-Passing Style* (CPS), o controle de fluxo não retorna implicitamente; uma função de "continuação"é passada como parâmetro, convertendo o espaço de pilha efêmero em estruturas estritamente controladas (DANVY, 2023).

- **Trampolim (*trampolining*):** Utilizado para evadir limites estáticos de recursão em interpretadores. A função encapsula seu estado em uma função anônima (*thunk*) e retorna para um laço controlador mestre. Para evitar explosão alocativa na memória, utiliza-se a defuncionalização para converter continuações em objetos algébricos estáticos (VERHOEFF, 2025).

3 TCO em Diferentes Linguagens de Programação

A adoção de TCO reflete prioridades distintas de design arquitetural, indo desde a garantia semântica até transformações robustas em tempo de execução.

Tabela 1: Suporte a TCO nas principais linguagens de programação

Linguagem	Suporte a TCO	Observação
Scheme	Obrigatório	Parte do padrão R^nRS; garantia assintótica PTR
Haskell	Nativo	Avaliação preguiçosa e continuações explícitas (join points)
Erlang	Nativo	Essencial para processos servidores indefinidos
JavaScript	Parcial	Suporte inconsistente; frequentemente convertido via trampolim (PARK et al., 2024)
C / C++	Via compilador	GCC/Clang otimizam; abordado via bibliotecas de pilha estendida para recursão abissal (MALKOV; ČUKI'C; ĐORĐEVI'C, 2026)
Go	Não nativo	Foco em iteração e eficiência de ponteiros (LION et al., 2022)
Java	Não suportado	Mitigado via conversão JIT de laços para métodos recursivos (INSA; SILVA, 2015)

3.1 C/C++ e Bibliotecas de Pilha Estendida

Em linguagens de foco em acesso nativo como C++, a ausência de TCO em recursões complexas resulta em falhas de segmentação. Trabalhos de fronteira contornam isso através de Bibliotecas de Pilha Estendida (ESL), replicando blocos transientes vitais sob demanda com sobrecarga zero, sem alterar a sintaxe abstrata do compilador principal (MALKOV; ČUKI'C; ĐORĐEVI'C, 2026).

3.2 Linguagens Gerenciadas e Transformação JIT

Linguagens como Java operam sob a JVM sem suporte nativo absoluto a TCO. Para contornar isso, algoritmos industriais transformam rotinas iterativas em lógicas recursivas; surprendentemente, o motor *Just-In-Time* (JIT) reconhece esses padrões, executando

inlining destrutivo que alcança desempenhos que rivalizam com as funções imperativas originais (INSA; SILVA, 2015).

4 Impactos Transacionais, Memória e GreenScore

A transformação de fluxos de controle não atua apenas sobre a velocidade (CPU), mas altera radicalmente a topologia da memória, impondo novos desafios métricos e ambientais.

4.1 O Custo Punitivo da Coleta de Lixo (GC)

Quando estruturas baseadas em pilha são substituídas por alocações no heap (via Trampolining ou CPS), o hardware transfere a gestão para o Coletor de Lixo. Modelagens mostram que sistemas altamente oscilatórios (como o algoritmo *Towers of Hanoi*) sofrem extrema degradação. Em arquiteturas massivas distribuídas, como Apache Spark, a criação cíclica efêmera força esvaziamentos na JVM que podem representar até 47% do tempo decorrido do processador isoladamente em cargas intensas de K-Nearest Neighbors.

4.2 Compilação JIT vs AOT e a Métrica GreenScore

A eficiência transcende os ciclos de clock e entra no âmbito da sustentabilidade. A literatura avalia as penalidades energéticas utilizando o índice GreenScore, modelado formalmente pela equação:

$$\text{GreenScore} = \alpha \times (\text{Energia}_{\text{norm}}) + \beta \times (\text{Carbono}_{\text{norm}}) + \gamma \times (\text{Tempo}_{\text{norm}}) \tag{1}$$

Análises exaustivas demonstram que em ambientes JIT (como PyPy), a performance lógica é frequentemente ofuscada pelo alto consumo de energia no aquecimento (*start-up*). Em contrapartida, integrações com compilação Estática *Ahead-Of-Time* (AOT) usando `ctypes` consolidam superioridade nas métricas ambientais, gastando frações irrisórias de energia e resíduos de carbono comparativamente (TURJA et al., 2025).

4.3 Microbenchmarking em Máquinas Virtuais Limitadas

Avaliar otimizações recursivas minúsculas impõe desafios devido à variação temporal da nuvem (*noisy neighbor*). Avaliadores superam essas barreiras utilizando filtragens matemáticas pelo método Wilcoxon Rank-Sum ou infraestruturas *Function-as-a-Service* (FaaS). Adicionalmente, para contêineres e arquiteturas da borda (*edge*), destacam-se ferramentas enxutas como o QUICK MB, que com apenas 25 MB isola as falhas puras de memória, I/O e processamento contínuo sem demandar matrizes artificiais pesadas (GARG; GUPTA, 2023).

Referências

BIERNACKA, M.; DANVY, O. A Syntactic Correspondence between Context-Sensitive Calculi and Abstract Machines. **BRICS Report Series**, v. 12, 2005.

DANVY, O. Folding left and right matters: Direct style, accumulators, and continuations. **Journal of Functional Programming**, v. 33, 2023.

GARG, Y.; GUPTA, B. QUICK MB: A MICRO BENCHMARKING TOOL FOR LITE VIRTUAL MACHINES. **International Journal of Progressive Research in Engineering Management and Science**, v. 03, 2023.

INSA, D.; SILVA, J. Automatic Transformation of Iterative Loops into Recursive Methods. **Information and Software Technology**, v. 58, 2015.

LION, David et al. Investigating Managed Language Runtime Performance: Why JavaScript and Python are 8x and 29x slower than C++, yet Java and Go can be Faster? In: PROCEEDINGS of the 2022 USENIX Annual Technical Conference (ATC '22). [S.l.]: USENIX Association, 2022. P. 835–852.

MALKOV, S. N.; ČUKI'C, I. Lj.; ĐORĐEVI'C, P. Ž. Enabling Deep Recursion in C++. **Computers**, v. 15, n. 1, 2026.

PARK, H. et al. Tail Call Optimization Tailored for Native Stack Utilization in JavaScript Runtimes. **IEEE Access**, 2024.

STEELE JR., Guy L. Debunking the "Expensive Procedure Call"Myth. In: PROCEEDINGS of the 1977 Annual Conference of the ACM. [S.l.: s.n.], 1977. P. 153–162.

TURJA, M. F. S. et al. Python Under the Microscope: A Comparative Energy Analysis of Execution Methods. **IEEE Access**, 2025.

VERHOEFF, T. The Olympiad Trap and an Old Trampoline. **Olympiads in Informatics**, v. 19, 2025.

WIKIPEDIA CONTRIBUTORS. **Tail call**. [S.l.: s.n.], 2025.